

Code Review of the main Branch of gpurec

Review carried out with four parallel review agents

11 August 2026

Abstract

This is a review of the `main` branch of `gpurec` as of commit `358a5b80`. It answers six questions: how much code there is, whether any files are bloated, whether the code is clean, whether the documentation is good, whether the naming conventions are consistent, and whether the directory tree is well organised. Every count and every file-and-line reference was produced by reading or running the code. Nothing in the repository was modified. Section 1 is a summary and a list of what to fix, in order; a reader who reads only that page will have the actionable content.

Contents

1	Summary and what to fix	2
2	What gpurec is, and how to read this review	4
2.1	What the software does	4
2.2	What this branch is for	5
2.3	Words used throughout	5
2.4	Where the three rules come from	6
2.5	What was not reviewed	7
2.6	Method	7
3	How much code is there?	7
3.1	There are no tests and no continuous integration	7
4	Are there bloated files?	8
4.1	<code>crates/gpurec-preprocess/src/lib.rs</code> — 2,401 lines	8
4.2	<code>gpurec/core/kernels/pi_forward.py</code> — 1,485 lines	8
4.3	<code>gpurec/core/kernels/wave_backward_kernels.py</code> — 1,583 lines	8
4.4	Long but justified — leave alone	9
4.5	The worst single function is not in a bloated file	9
5	Is the code clean?	9
5.1	What came back clean	9
5.2	Three Rust call sites raise an exception nobody can catch	9
5.3	986 lines have no caller	10
5.4	A stale default that is 21 times too small	10
5.5	The same settings in three or four places	11
5.6	Six files take settings from the environment	11
5.7	Clamping	12
5.8	Duplication	12
5.9	Smaller items	13

6	Is the documentation good?	14
6.1	The shipped example fails, and the rule it breaks is written nowhere	14
6.2	Sixteen pointers to deleted documentation	14
6.3	Docstring coverage	15
6.4	Comments in the kernels explain the wrong layer	16
6.5	The READMEs	16
6.6	The command-line tool is undocumented	17
6.7	What a new user cannot work out	17
7	Are the naming conventions consistent?	17
7.1	Short names nobody has written down	17
7.2	Two names that mislead	18
8	Is the directory tree well organised?	18
8.1	The intended layering, and where it is violated	18
8.2	The workaround, and a trap in removing it	19
8.3	Six package directories have no <code>__init__.py</code>	19
8.4	The private-module convention is not enforced	20
8.5	<code>fit/</code> and <code>solver/</code> overlap	20
8.6	The two loose files at the package root	20
8.7	The public surface, and one packaging gap	20

1 Summary and what to fix

Verdict

This branch is not yet fit to be handed to an outside user, and the reason is not the code — it is everything around the code. The discipline inside the source files is unusually good: no leftover markers, no commented-out blocks, no stray debug output, no unsafe Rust, and naming conventions that hold across 358 functions. What fails is the surface a new user touches. The one shipped example does not run, the rule needed to make it run is written down nowhere, there are no tests, and sixteen places in the source point at documentation files that were deleted.

For reading and for small local fixes, this is a comfortable codebase. For any change that moves a number, it is not, because there is nothing on this branch that can tell you whether you changed an answer.

The five findings that matter

- 1. The shipped example does not run, and the rule it breaks is undocumented.** A gene leaf must be named `SPECIES_gene`; the code splits on the first underscore. That rule appears in no README, docstring or comment. The shipped example uses leaves called `a` and `b`, so the headline snippet in the README fails. A second file, `examples/tiny/gene.map`, is presented by two READMEs as the thing that maps genes to species and is read by nothing. (Section 6.1.)
- 2. There are no tests and no automated checks, and three files claim otherwise.** The most damaging is a comment naming a “permanent guard test” that would catch the settings file drifting out of step with the Python code it mirrors. That test is not on this branch, so the two copies are free to disagree silently. (Section 3.1.)
- 3. One stale default is wrong by a factor of 21.** A function declares `neumann_terms=3` where the configured value is 64. Production callers always pass it explicitly, so it never fires in normal use — it fires for whoever calls that function by hand. (Section 5.4.)

4. **The same settings live in three or four places, and six files take settings from shell environment variables.** Two of those environment variables are read *before* the argument the caller explicitly passed, so a deliberate choice in code is silently overridden by something invisible. (Sections 5.5 and 5.6.)
5. **986 lines have no caller, and two settings classes sit in the wrong package.** The misplaced settings classes are why the settings package imports the model package, why the lowest-level kernels import the model package, and why one import had to be deferred inside a function with a nineteen-line comment apologising for it. (Sections 5.3 and 8.)

What to fix, in order

Effort is a rough order of magnitude for one engineer who knows the code.

Table 1: Recommended order of work. “Owner” marks items that need a decision from whoever owns the branch rather than a code change.

#	What to do	Effort	Owner?
1	Make the shipped example run: rename the example leaves to <code>A_a</code> and <code>B_b</code> , delete <code>examples/tiny/gene.map</code> , write the underscore rule into <code>README.md</code> and <code>examples/tiny/README.md</code> , and fix the family-listing path bug at <code>gpurec/fit/genewise_fit.py:79-80</code> so the command-line route works too. Not closed until the README snippet runs end to end.	hours	
2	Fix <code>neumann_terms=3</code> at <code>gpurec/core/kernels/wave_backward.py:673</code> .	minutes	
3	Replace the three <code>.unwrap()/.expect()</code> calls in the Rust with the crate’s own typed error, so a malformed input file raises a normal Python exception instead of one that <code>except Exception</code> cannot catch.	hours	
4	Decide the test story. Bring <code>tests/</code> onto this branch. The alternative — deleting the three false claims — is cheaper but leaves every item below unverifiable, so it is not recommended.	—	yes
5	Move <code>SolverOptions</code> out of <code>gpurec/api/</code> and <code>PenaltyOptions</code> out of <code>gpurec/solver/</code> , both into <code>gpurec/config/</code> . Removes the deferred-import workaround and both layering inversions. Do this <i>before</i> adding any <code>__init__.py</code> to <code>api/</code> — see Section 8.2.	hours	
6	Replace the two <code>clamp</code> calls at <code>gpurec/fit/genewise_fit.py:328</code> and <code>:335</code> with the <code>torch.maximum</code> form already used at <code>gpurec/fit/global_fit.py:128</code> and <code>:136</code> .	minutes	
7	Make <code>gpurec/fit/newton_cg.py</code> read <code>NewtonOptions</code> the way <code>gpurec/fit/genewise_fit.py:49-56</code> reads its settings. Removes 31 duplicated literals and closes two drifts the code already admits to.	hours	
8	Move the six environment variables into settings objects, and give the four batch-planning knobs a dataclass alongside <code>SolverOptions</code> . One of them is read at import time to set a GPU launch parameter, so this is not purely mechanical.	days	

#	What to do	Effort	Owner?
9	Triage the 986 dead lines <i>with the author</i> . 456 of them are research code that says in a comment it is deliberately not wired in; that is not a reviewer’s call.	hours	yes
10	Restore <code>docs/latex/kernel_mathematics.tex</code> . Deleting the 16 dangling pointers instead is cheaper, but that file is the only thing that ever explained the recurrences and eight kernel modules still send readers to it.	—	yes
11	Write a class docstring for <code>GeneReconModel</code> covering its twelve constructor arguments, the rate order (duplication, loss, transfer), the base-2 logarithm convention, and that the returned loss is in bits.	hours	
12	Add READMEs for <code>solver/</code> , <code>fit/</code> , <code>config/</code> and <code>cli/</code> , and add those four directories to the root repository map.	hours	
13	Extract the 109-line duplicated Triton block in <code>wave_backward_kernels.py</code> into a device helper, following the existing <code>_load_event_log_probability</code> precedent.	hours	
14	Split the three bloated files along the boundaries in Section 4. Purely structural, nothing depends on it, and therefore last.	days	

The one thing behind most of this

`main` is produced by hand-deleting things from `dev`. The recent history reads “Remove all docs from main”, “Remove more non-essential docs from main”, “Remove planning docs from main”, “Remove `gpurec/bench` from main”, “Remove `gpurec/solver/_golden` test fixture from main”.

Most of the defects in this review are debris from that process rather than independent mistakes: the sixteen pointers to deleted documents, the three false claims that tests exist, the settings guard that cannot fire, the `notebooks/` directory the README advertises, the `pytest` configuration with nothing to configure. Fixing them one by one leaves the process intact, and the next strip will produce a fresh set.

The durable fix is to generate `main` from `dev` with a script, and to have that script fail when it leaves a reference dangling. That recommendation is not on the numbered list above because it is a change to how the project is released, not to the code — but it is worth more than items 10 through 14 combined.

2 What `gpurec` is, and how to read this review

2.1 What the software does

`gpurec` takes one fixed *species tree* — the family tree of a set of organisms — together with many *gene families*, each being the set of copies of one gene across those organisms. For each family it computes how likely that family’s history is, given rates at which genes are duplicated within a lineage, lost, and transferred sideways from one lineage into another. Fitting those three rates to the data is what the package is for. The arithmetic runs on a graphics card.

The project’s own summary, at `README.md:1-8`, calls it “an experimental GPU-accelerated likelihood engine for gene-tree/species-tree reconciliation under a DTL-style model”, which returns “a scalar negative log-likelihood in bits”.

Rates can be shared three ways, selected by a `mode` setting: one set of rates for everything (`global`), one per species (`specieswise`), or one per gene family (`genewise`).

2.2 What this branch is for

`main` is the stripped branch. It carries 96 files; `dev` carries those plus 252 more, including 63 test files and 75 documentation files. The review assumes `main` is what an outside user would install, because that is the only role a branch in this state could serve — but that assumption is the reviewer’s, not something the repository states, and it is the single biggest lever on how severe the documentation findings are. If `main` is instead an internal artefact that nobody clones, findings 1 and 2 drop sharply in importance and little else changes.

2.3 Words used throughout

Defining these is not decoration. One of this review’s own findings is that the codebase uses short names nobody has written down, so the review had better not do the same thing.

Table 3: Terms used in this document. “Where it comes from” says whether the meaning is stated in the repository or was inferred from the code.

Term	Meaning	Where it comes from
reconciliation	Explaining a gene family’s tree as a history inside the species tree: at each point, saying whether a gene duplicated, was lost, was transferred, or simply followed a speciation.	standard usage; the README uses it undefined
clade	A group of gene copies that could form one subtree. <code>gpurec</code> works over a distribution across possible clades rather than one fixed gene tree.	README.md:15–16
wave	One level of the species tree, processed as a batch. The Rust preprocessor groups work into waves so a whole level runs in one kernel launch.	README.md:32–33
the wave step	The kernel that advances the per-clade, per-species quantities by one level of the species tree.	inferred from the code
the gene-split reduction	Combining the two children of a gene node into their parent, summing over the event that joined them. “DTS” in file names.	<code>core/inference/</code> README.md:7
the extinction step	Computing, for each species, the probability that a lineage there leaves no observed descendants. Called E; it is a fixed point, solved by iterating to convergence.	README.md:35
DTS	Duplication, transfer, speciation — the events combined in a gene-split reduction. Written out in exactly one place in the repository.	<code>core/inference/</code> README.md:7
Pi, Pibar	The reconciliation likelihood per clade and species, and its transfer-weighted counterpart, propagated wave by wave.	README.md:36, informally
E, Ebar	The extinction probabilities above, and their transfer-weighted counterpart.	README.md:35, informally
theta	The fitted parameters: base-2 logarithms of the three rates, in the order duplication, loss, transfer.	<code>cli/reconcile.py</code> :31

Term	Meaning	Where it comes from
bits vs. nats	Two units for a log-likelihood: base-2 logarithm versus natural logarithm. The library works in bits; the command-line tool prints nats.	<code>cli/_common.py:8-10</code>
vjp	Vector-Jacobian product: the reverse-mode derivative step, i.e. what a gradient pass computes. Never expanded in the repository.	inferred
jvp	Jacobian-vector product: the forward-mode derivative. Never expanded.	inferred
hvp	Hessian-vector product: the second-derivative step.	<code>solver/hvp/__init__.py:1</code>
Newick, ALE	Two text formats for trees that the Rust preprocessor reads.	<code>crates/\protect\global\let\T1\textellipsis.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font\T1\textellipsis/lib.rs</code>

The four passes. Each of the three pieces of work above exists in four versions, and the file suffixes encode which: a *value* pass (no suffix) computes the likelihood; a *gradient* pass (`_backward`) computes its first derivative in reverse mode; a *forward-derivative* pass (`_tangent`) computes a first derivative in forward mode; and a *second-derivative* pass (`_so`, for “second order”) computes curvature. All four exist because the package offers Newton-type fitting, which needs second derivatives, and computing those efficiently needs the forward-mode pass as an ingredient. This three-by-four grid is used later as an argument for how to split two files.

2.4 Where the three rules come from

Three rules are used below to judge the code:

1. no default values on function parameters;
2. no environment variables for configuration;
3. never clamp a value.

These are the project owner’s stated working rules, given to the reviewer directly. They are not written down anywhere in this repository. That is worth saying plainly, because every other claim in this document carries a file and a line number and these three cannot. A reader who does not accept these rules should read Sections 5.4, 5.6 and 5.7 as observations rather than as findings — although the first of them, the stale default, is a correctness hazard on any standard.

The reasoning the owner gives for rules 1 and 2 is the same in both cases: one value should live in one place. A default is a second home for a number, and it drifts from the settings file

that is supposed to own it, silently, because the caller that always passes the real value hides the stale default. Section 5.4 is that failure, already happened.

2.5 What was not reviewed

This review covers structure, cleanliness, documentation and naming. It does **not** cover: whether the mathematics is correct; whether the numerical results are accurate; performance; input validation beyond the three Rust crash sites noted; dependency pinning; or licensing. No benchmark was run and no likelihood was checked against a reference implementation.

2.6 Method

The branch was checked out into a separate git worktree so the working copy on `dev` was untouched. Four review agents worked in parallel on disjoint parts of the tree: the GPU kernels and the public model wrapper; the solver, fitting, command-line and settings code together with the Rust crates; the documentation; and the naming and layout.

Their findings were then re-checked directly rather than taken on trust. Every claim reproduced in Section 1, plus the file and line counts, the dead-module list, the environment variables, the clamp sites and the import graph, was verified by re-running the search or the program. The shipped example was run through the real Rust preprocessor. That re-check overturned two agent claims, both corrected here: the Rust crash sites do not abort the process (Section 5.2), and the deferred-import workaround is caused by `PenaltyOptions`, not by `SolverOptions` (Section 8). Counts that were not re-derived — the duplication measurements and the docstring percentages — are marked where they appear.

3 How much code is there?

The branch holds **24,829 lines across 93 files**, in a tree of 96 tracked files. The three not counted are `.gitignore` and the two `Cargo.lock` files.

Table 5: Size of the branch. Line counts are raw lines including blanks and comments.

	Files	Lines
Python	64	18,007
Rust	5	6,149
Markdown (all READMEs)	16	506
Settings and example data	8	167
Counted	93	24,829
Not counted (<code>.gitignore</code> , two <code>Cargo.lock</code>)	3	—
Tracked on the branch	96	

The Python figure of 64 files is 63 under `gpurec/` plus `setup.py` at the root. Later sections that scan only the package report 63; both numbers are used below and are labelled where they appear.

3.1 There are no tests and no continuous integration

There is no `tests/` directory, no `conftest.py`, no `.github/` directory and no YAML file of any kind on this branch. Three files say otherwise.

Table 6: Claims on this branch that tests exist.

Location	Claim
pyproject.toml:36	testpaths = ["tests"], plus three test markers
README.md:162	tells the reader to run <code>pytest</code>
gpurec/config/defaults.toml:5	names a “permanent guard test” <code>tests/test_config_toml.py</code>

The third is the damaging one. That guard is meant to fail if `defaults.toml` drifts from the Python dataclass it mirrors. It is not here, so it cannot fire, and Section 5.5 shows the drift it was supposed to catch.

4 Are there bloated files?

Three files are bloated in the sense that one file is doing several unrelated jobs. Two more are long but cohesive and should be left alone.

4.1 crates/gpurec-preprocess/src/lib.rs — 2,401 lines

Five separate jobs, each already in a contiguous block, so the split is mechanical.

Table 7: Proposed split of the largest Rust file. “Current lines” are line ranges in the file as it stands.

Proposed file	Current lines	Size	What it does
<code>ale_format.rs</code>	922–1480	559	reads the ALE file format
<code>family_ccp.rs</code>	1870–2297	428	the <code>FamilyCcp</code> Python class
<code>amalgamation.rs</code>	757–921, 1678–1869	357	builds clade distributions from gene trees
<code>newick.rs</code>	153–409	257	parses gene trees in Newick format
<code>bitset.rs</code>	1580–1677	98	15 small bit-vector helpers
<code>lib.rs</code> remainder	—	~620	the 4 Python entry points and orchestration

There is independent evidence this file was left behind when its siblings were cleaned up. The three sibling modules use the crate’s typed error type **37 times between them and plain string errors zero times**. This file still uses plain string errors **27 times** and the typed error twice.

4.2 gpurec/core/kernels/pi_forward.py — 1,485 lines

Two jobs: the wave step (lines 91–695 and 1080–1325) and the gene-split reduction (lines 699–1078 and 1328–1485, about 530 lines).

The codebase itself argues for the split. In the three-by-four grid described earlier, the derivative versions of exactly this code are *already* separated — `dts_tangent.py` and `dts_so.py` hold the gene-split reduction, `wave_tangent.py` and `wave_so.py` hold the wave step. Only the value pass and the gradient pass keep the two merged.

4.3 gpurec/core/kernels/wave_backward_kernels.py — 1,583 lines

Three jobs. The grouping is unambiguous because one can read off which wrapper in `wave_backward.py` launches which kernel.

Table 8: Proposed split of the largest gradient file. “Kernel starts at” gives the line number where each kernel is defined.

Group	Lines	Kernel starts at
wave-step gradient	~883	12, 45, 334, 531, 624
gene-split gradient	~495	906 (one kernel, running to 1400)
transfer-complement gradient	~178	1404, 1440, 1480

4.4 Long but justified — leave alone

`crates/gpurec-preprocess/src/scheduler.rs` (1,361 lines, 44 functions) and `batch_planning.rs` (768 lines, 20 functions) are cohesive: every function serves one job, wave scheduling and batch packing respectively.

The individual Triton kernels being long is also not a defect. Triton offers no cheap function call inside a kernel body, so a 500-line kernel is inherent to the tool. The three longest are 495, 458 and 322 lines.

4.5 The worst single function is not in a bloated file

`make_exact_hvp_single` in `gpurec/solver/hvp/exact.py` runs from line 191 to line 892 — **702 lines** — of which the returned inner function is **458 lines in one body** (lines 435–892). The other four top-level functions in that 1,000-line file total 148 lines, so the file is fine and this one function is not. It already carries three internal section markers, at lines 372, 817 and 891, at natural split points. As written it cannot be tested or profiled piecewise.

Two other Python functions exceed 250 lines: `wave_backward.py:198` (466 lines) and `api/_implicit_grad.py:154` (394 lines).

5 Is the code clean?

Inside the source files, yes. The problems are in how settings and dead code are managed, not in how the code is written.

5.1 What came back clean

Zero TODO, FIXME, XXX or HACK markers anywhere; zero bare `except:` clauses; zero mutable default arguments; zero commented-out code; zero stray debug prints (all 55 `print()` calls are behind a verbosity flag, are a command-line message, or go to standard error); and in the Rust, zero `unsafe` blocks and zero explicit `panic!`, `unreachable!`, `todo!` or `unimplemented!` macros across 6,149 lines. Nesting nowhere exceeds five levels.

That is a better result than most codebases return, and it is why the verdict separates the code from its surroundings.

5.2 Three Rust call sites raise an exception nobody can catch

Three places in the Rust use `.unwrap()` or `.expect()` on a value that a malformed input file can make absent:

```
crates/gpurec-preprocess/src/lib.rs:1109
    let observations = ale.observations.expect("validated observations");
crates/gpurec-preprocess/src/lib.rs:1806
    let idx = *split_index_map.get(&canon).unwrap();
crates/gpurec-backtrack/src/lib.rs:413
    let sampler = self.recipient_cache[clade].as_ref().unwrap();
```

All three are reachable from Python. **What happens is not a crash of the process**, and an earlier draft of this review said it was. Neither `Cargo.toml` sets `panic = "abort"`, so the default unwinding behaviour applies and the Python binding layer catches the panic at the boundary. The user sees `pyo3_runtime.PanicException`.

That is still a real defect, for two reasons. First, that exception derives from Python’s `BaseException` rather than `Exception`, so an ordinary `try/except Exception` around a file-loading call will not catch it — a caller processing a thousand families cannot skip the one bad file and continue. Second, it bypasses the crate’s own well-built typed error, which already knows how to turn a failure into the right Python exception class. The fix is to use that error type, and it is item 3 on the list.

5.3 986 lines have no caller

Each was checked by searching the whole tree — Python, Rust, Markdown and plain text — for any reference outside the defining file.

Table 9: Code with no caller anywhere in the repository.

Item	Lines
<code>gpurec/solver/curvature/genewise.py</code> — the whole file	456
<code>gpurec/distributed.py</code> — the whole file	112
<code>gpurec/fit/baselines.py</code> — the whole file	62
<code>newton_tr</code> and <code>newton_cg</code> , <code>fit/newton_cg.py:274-435</code>	162
<code>receiver_information</code> , <code>solver/curvature/receiver.py:195</code>	75
<code>origination_information</code> , <code>solver/curvature/origination.py:143</code>	70
<code>plan_family_batches</code> , <code>core/scheduling/batching.py:394</code>	49
Total	986

The first entry says so itself: `solver/curvature/genewise.py:60` carries the comment “TEST-ONLY / NOT wired into the fit ...do NOT wire it into the fit”. That is 456 of the 986 lines, and it is why item 9 on the list says to triage this with the author rather than delete it.

A distinction worth keeping: a further body of code is unreachable from the two entry points but is *intentional* research surface. `gpurec/fit/dtl_fit.py:73` deliberately raises `NotImplementedError` for one fitting mode and names the hand-import entry points in the error message. That is the right pattern. What the 986 lines above lack is either a caller or such a pointer.

5.4 A stale default that is 21 times too small

`gpurec/core/kernels/wave_backward.py:673` declares `neumann_terms=3`. The configured value is **64**, in two places: `gpurec/api/solver_options.py:12` and `gpurec/config/defaults.toml:22`.

That setting controls how many terms of a series are summed when solving for the gradient — step 7 of the README’s execution path, “a matrix-free Neumann-series solve for the E-adjoint linear system”. Both production callers pass the value explicitly, which is exactly why the stale **3** has gone unnoticed: it never runs in production. It runs for whoever calls that function by hand and gets a gradient built from 3 terms instead of 64, with no warning.

How wrong is the answer? This review does not know, and that is a real gap. The size of the error depends on how fast that series converges for this model, which was not measured — measuring it was outside the scope stated above. What can be said without measuring: the project chose 64 and wrote it into the settings file twice, so 3 is not a value anyone deliberately selected, and a silent 21-fold reduction in solver work is not something a caller should receive by accident. Anyone fixing item 2 should take ten minutes to compare the two gradients on a small family and record the answer, because it converts this from a hazard into a known quantity.

A second instance of the same shape: `gpurec/api/_implicit_grad.py:82` hardcodes `max_iter: int = 128`, restating a value already declared at `gpurec/api/solver_options.py:17`.

The measurement. Across `solver/`, `fit/`, `cli/`, `optimization.py` and `distributed.py` there are **446 defaults on 817 parameters, or 55 %**. Of those 446: **219** are `None` sentinels, **142** are bare numeric literals, 44 are booleans, 20 are strings and 21 are other expressions.

The 142 numeric literals are the ones that matter, because a number is what drifts. The `None` sentinels are a different thing wearing the same clothes: most of them mean “the caller did not supply this”, and several are the good pattern where the function then reads the real value from the settings object — one value, one place, which is what the rule is protecting. The booleans, strings and other expressions were not individually assessed.

The worst offenders by numeric literal are `fit/newton_cg.py` (31), `fit/optimize.py` (28), `fit/global_fit.py` (15) and `fit/genewise_fit.py` (13).

The rule *is* followed in one place and it works: `gpurec/fit/specieswise_fit.py:29` makes the regularisation weight a required argument and raises a clear error explaining that a weight of zero is the unregularised fit and is deliberately not a default.

5.5 The same settings in three or four places

`gpurec/config/defaults.toml` is **never read at runtime** unless the user passes `--config`. `gpurec/config/gpurec_config.py:210` returns the plain Python dataclass values with no file access. So the file is a hand-maintained copy of 45 numbers with nothing checking it — the guard test it names is not on this branch.

The Newton solver’s settings exist three times over: as 21 fields in `gpurec/config/newton.py`; repeated at `defaults.toml:39-59`; and re-typed as function-parameter defaults at `fit/newton_cg.py:49-52`, `:274-276` and `:366-368` — a file containing **zero** references to the settings class.

The code already admits the drift. `gpurec/config/newton.py:35-39` says two fields are “RESERVED: validated here but not yet read by `fit/newton_cg.py` (FD literal is inline)”. `gpurec/fit/optimize.py:517-519` notes the same setting standing at 8 in one place and 40 in another.

A fourth partial mirror sits at `gpurec/config/gpurec_config.py:159` and `:175`, re-typing solver values already identical to the dataclass defaults. And four batch-planning settings have **no settings home at all**, appearing as defaults in five signatures: `family_chunk_size=300`, `clade_budget=315,000`, `batch_packing="depth_first_fit"`, `max_wave_size=8192`.

Credit where due: `fit/genewise_fit.py:49-56` and `fit/map_cv.py:46` *derive* their solver settings from the factory methods instead of retyping them. That is the pattern item 7 should copy.

5.6 Six files take settings from the environment

Table 10: Environment variables used as settings.

Location	What it does
<code>gpurec/core/memory_policy.py:69-70</code>	The worst two. Read as <code>os.environ.get(NAME, str(default_fraction))</code> — the shell variable is the first choice and the argument the caller explicitly passed is only the fallback. A deliberate setting is silently overridden.
<code>gpurec/fit/genewise_fit.py:245-374</code>	<i>Writes</i> a variable before the fit, pops it mid-loop to force a cold re-check, restores it in a <code>finally</code> . Nine statements.

Location	What it does
<code>gpurec/core/kernels/wave_tangent.py:24</code>	A GPU launch parameter read at import time, so it cannot appear in a settings file or in the record of a run.
<code>gpurec/solver/hvp/exact.py:207,233</code>	Two numeric solver knobs.
<code>gpurec/api/_execution.py:138,310</code> and <code>solver/hvp/exact.py:62</code>	Switch a real behaviour: whether the previous call's gradient is reused as a starting guess.
<code>gpurec/distributed.py:37-41</code>	Multi-process settings. (This file is dead; see Section 5.3.)

Two observations. `gpurec/api/solver_options.py:27-34` documents a sibling setting and states “config-only, no env var” — so this is a known migration that stalled partway. And `gpurec/config/memory.py:28-29` documents the memory variables in comments while the `dataclass` field beside them holds the same value, which is two homes for one setting.

One use is *not* a violation and should be kept: `gpurec/core/native.py:15` reads a path variable to locate the compiled Rust library, and documents it in the error message at line 53. That is a fact about where a build artefact lives, not a setting.

The Rust uses no environment variables.

5.7 Clamping

Two real violations, and in both cases the compliant form already exists two files away:

- `gpurec/fit/genewise_fit.py:328` — `e.clamp(min=mu)` on curvature eigenvalues. `gpurec/fit/global_fit.py:128` does the same job with `torch.maximum`.
- `gpurec/fit/genewise_fit.py:335` — `dn.clamp(min=trust)`. `gpurec/fit/global_fit.py:136` uses `torch.maximum`.

One case needs a decision rather than a fix. `gpurec/optimization.py:95` (`clamp_log_rate_`) clamps the fitted rates onto their allowed range. It is deliberate, it is public API, and five call sites depend on it. The options are: keep it and record in a comment that this is the one sanctioned clamp; or replace it with a projection that reports when it binds, so a rate pinned at its bound is visible rather than silent. The second is better if a bound-hitting rate would otherwise be mistaken for a converged fit.

Five further uses were checked and are legitimate: an array-index bound at `core/parameters/extract_parameters.py:137` (whose comment records that the old value-clamp was deliberately removed); the denominator of a reported diagnostic at `wave_backward.py:535`; and three guards on a variance before a square root, in `solver/curvature/`.

5.8 Duplication

Measured by the review agents; the two largest were re-checked directly.

109 byte-identical lines in one file. In `gpurec/core/kernels/wave_backward_kernels.py`, the block at lines 156–264 and the block at lines 723–831 are byte-for-byte identical. Counting shorter runs, the two kernels at lines 45 and 624 share **156 identical lines** across four runs; both kernels are about 280 lines, so this is over half of each. The duplicated block is shared setup: loading the event constants, gathering the two child rows, forming the loss and transfer terms.

This is clearly fixable, because the project already does it elsewhere: `pi_forward.py` defines shared device helpers and two other kernel files import one across file boundaries.

About a third of a 476-line file is a copy of another part of it. In `gpurec/api/_execution.py`, two functions of 102 and 104 lines share **94 identical lines**, differing in exactly four things: the name, an extra flag, an up-front error check, and which of two loss functions they call. One level up, two more functions share 42 of about 55 lines. Together that is roughly 150 of the file’s 476 lines. The environment-variable block noted earlier exists in duplicate at lines 138 and 310 for this reason.

178 near-copied lines across three curvature files. Counting a pair of lines as a copy when they are at least 70 % character-identical after normalising whitespace — a fuzzy measure, so treat these as indicative:

Table 12: Near-copies within `solver/curvature/`. “Share” is of the smaller of the two function bodies.

Pair	Lines	Share
<code>receiver.py:273</code> vs <code>origination.py:216</code>	50	85 %
<code>receiver.py:195</code> vs <code>origination.py:143</code>	45	73 %
<code>origination.py:216</code> vs <code>genewise.py:223</code>	36	60 %
<code>receiver.py:147</code> vs <code>origination.py:108</code>	26	87 %
<code>origination.py:108</code> vs <code>genewise.py:184</code>	21	70 %
Total	178	

Those three files total 1,083 lines, so this is about 16 %. Because `origination.py` appears in three rows, the 178 counts each *pairing*, not each distinct line — the number of lines that could be deleted is smaller than 178 and was not measured. A sixth pairing, of 9 lines, sits between `global_fit.py` and `genewise_fit.py` and is discussed next.

Credit where due: `solver/curvature/gauge.py` already holds the shared core, and `origination.py:38` imports helpers from `receiver.py` rather than copying them. One round of de-duplication has happened; the wrapper layer is what remains.

One of these is a behaviour difference, not repetition. `fit/genewise_fit.py:326` builds the small curvature matrix analytically, while `fit/global_fit.py:120-125` rebuilds the same matrix by finite differences — three extra forward-and-backward passes per refresh — even though its own docstring says it runs “the SAME recipe”. Worth resolving independently of the duplication.

Two more pairs in the kernels. `wave_tangent.py:563-648` and `:650-734` are two 85-line launchers with 73 lines identical. `:28-272` and `:276-487` share 130 identical lines, the second being the first wrapped in a repeat loop.

What is *not* duplicated. The four passes are genuinely different code, not variants of one thing. The three `e_step` files share only their input loading; the mathematics differs. The real duplication is *within* files, not across the passes.

5.9 Smaller items

- **Nine dead defensive lookups** of the form `getattr(static, "field", fallback)` — at `api/_execution.py:23,73,138,310`, `api/_autograd.py:36,70`, `core/inference/solver.py:42,85` and `core/backtracking/input.py:57` — all name fields that *are* declared on the dataclass at `api/_batch_state.py:13-36`. The fallback can never fire, and it is worse than nothing: a mistyped field name would silently return the fallback instead of raising.

- **A hardcoded personal path in library code.** `gpurec/fit/map_cv.py:173` contains an absolute path under one developer's home directory, inside a self-test wired to a `--main__` block. That directory does not exist even in this checkout.
- **One raw print to standard output** at `gpurec/api/model.py:177-182`. The information is legitimate; it belongs in a warning or a logger so callers can silence or capture it.
- **A magic iteration count with no home**, 400 at `gpurec/api/model.py:366`, with no comment explaining it.
- Two stale path comments: `gpurec/solver/penalties.py:1` names a directory that does not exist, and `gpurec/config/newton.py:4` points at a file that became a package in commit 5a5b0f33.

6 Is the documentation good?

No. Two problems dominate: the example does not run, and sixteen pointers lead to deleted files.

6.1 The shipped example fails, and the rule it breaks is written nowhere

Run against the real Rust preprocessor:

```
preprocess_dataset(examples/tiny/species.nwk, [examples/tiny/gene.nwk])
-> ValueError: species "a" not found for gene leaf "a"
```

`gpurec` maps a gene to its species by splitting the leaf name on the **first underscore** — `split_once('_')` at `crates/gpurec-preprocess/src/lib.rs:516`, and again at line 1657. That is the only mechanism and it is documented nowhere: not in a README, not in a docstring, not in a comment.

`examples/tiny/gene.nwk` contains `(a:1,b:1)`; with no underscores, so the headline example at `README.md:47-66` fails on the only dataset the repository ships. Renaming the leaves to `A_a` and `B_b` was tested and gets past the leaf-mapping stage; **whether the full README snippet then runs to completion was not confirmed**, which is why item 1 on the list is not closed until someone runs it end to end.

Two further problems in the same example:

- `examples/tiny/gene.map`, containing `A:a` and `B:b`, is a decoy. Searching every Python and Rust file finds **nothing that reads it**, yet `examples/README.md:13-14` and `examples/tiny/README.md:12-13` both present it as the thing that assigns genes to species. A new user will edit that file and wonder why nothing changes.
- `examples/tiny/families.txt` names its gene tree as `gene.nwk`, and the resolver at `gpurec/fit/genewise_fit.py:79-80` takes that string verbatim without joining it to the listing file's own directory. So it resolves relative to the current working directory, and the command-line route into the same example cannot find the file from the repository root.

6.2 Sixteen pointers to deleted documentation

All of `docs/` was removed from this branch; the references to it were not.

Table 13: References to documentation files that are not on this branch.

Missing target	Refs	Referenced from
docs/latex/kernel_mathematics.tex	9	core/kernels/README.md:8 (a clickable link), plus line 1 of six kernel modules, <code>dts_tangent.py</code> :191 and <code>wave_backward.py</code> :3
docs/config_convention.md	3	<code>fit/optimize.py</code> :484, <code>fit/map_cv.py</code> :77, <code>fit/genewise_fit.py</code> :60
docs/pi_storage.md	1	api/README.md:12
docs/centered_state_contract.md	1	core/pi_state.py:4
docs/superpowers/\protect.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font	2	core/parameters/fraction_missing.py:6, <code>fit/specieswise_fit.py</code> :5

The first row is why item 10 recommends restoring rather than deleting. `gpurec/core/kernels/README.md`:8–9 states: “*The canonical equations ... are in [that file]. Python docstrings are intentionally limited to interface and launch behavior.*” That was a sound policy while the file existed. Deleting it turned the policy into a hole — six kernel modules now carry one-line docstrings whose whole content is “see this file”.

Three more dead pointers sit in `pyproject.toml`: a plan file at line 12, an experiments script at line 25, and `solver/cg.py` at line 17 (the file is `solver/krylov.py`).

6.3 Docstring coverage

Measured by a review agent over all 64 Python files. “Public functions” means functions and methods whose name does not begin with an underscore.

Table 14: Docstring coverage across the branch.

	Documented	Share
Module docstrings	43 of 64	67 %
Public functions and methods	126 of 239	53 %
Private functions	58 of 119	49 %
Classes	11 of 17	65 %

Coverage is sharply split and *inversely* correlated with README coverage: the directories with no README are the best documented in code, and the directories with READMEs are the worst.

Table 15: Best and worst documented files. Fractions count public functions only, so a large kernel file can show a small denominator: the Triton kernels themselves are private, and only the Python launch wrappers are counted.

Well documented	Barely documented
<code>config/gpurec_config.py</code> 10 of 10	<code>core/kernels/pi_forward.py</code> 0 of 3, in 1,485 lines
<code>solver/curvature/receiver.py</code> 8 of 10	<code>core/kernels/e_step.py</code> 0 of 5, in 605 lines
<code>solver/curvature/origination.py</code> 7 of 8	<code>core/inference/forward.py</code> 0 of 1 — the forward pass itself
<code>solver/penalties.py</code> 8 of 10	<code>api/_implicit_grad.py</code> 0 of 2, in 670 lines
<code>fit/optimize.py</code> 5 of 6	<code>api/_autograd.py</code> 0 of 4 functions, 0 of 2 classes

The main public class has no docstring at all. `GeneReconModel` at `gpurec/api/model.py:34` — the one object the README tells the reader to use — has no class docstring, no constructor docstring, no `forward` docstring, and its module has none either. Its constructor takes twelve keyword arguments, three of which are unexplained magic numbers. The three methods the README advertises are all undocumented. That is item 11.

A concrete bad example. `core/kernels/pi_forward.py:1156` is a public launch wrapper with **24 parameters**, no docstring, no type hints and no shape annotations. A reader cannot tell what any tensor’s shape is without tracing a caller. Two neighbouring wrappers have 21 and 22 parameters.

Two good examples worth copying. `fit/global_fit.py:1-19` states the parameter vector and its shape, the objective, the recipe, *why* families are never dropped in that mode, and the measured speedup. `solver/curvature/receiver.py:1-30` states the invariance, why it makes the curvature matrix singular, and what projection removes it.

6.4 Comments in the kernels explain the wrong layer

Comment density in `core/kernels/` runs 0–6%, against 13% in `solver/hvp/exact.py`. The bare numbers are less interesting than the pattern, so the table is omitted; what matters is *what* is commented.

The comments that exist are good, and they cover numerical and hardware concerns: why an index is widened to 64-bit to avoid overflow (`pi_forward.py:227-228` and two others), why a row offset is absorbed lazily (`pi_forward.py:441-443`), and an excellent five-line explanation of a cross-thread lost-update race at `e_step.py:216-220`.

What is absent is the model. Nowhere in the kernel directory is there a statement of what the extinction probability or the reconciliation quantities are, or what the recurrence computes. The fixed point at the heart of the model, `e_step.py:19`, has no docstring and two comments, neither about the recurrence.

Two mitigations: the names are unusually self-documenting, and `wave_backward_kernels.py:912-945` annotates each of about 30 pointer arguments inline with its shape and meaning. That block is the best tensor documentation on the branch and is the right template for the `pi_forward.py` wrappers.

Note that density alone is not the standard here. `solver/hvp/exact.py` has the highest density on the branch and also contains the 702-line function criticised in Section 4; good comments do not excuse structure, and the point is that the kernels lack the *kind* of comment that would help, not that they lack volume.

6.5 The READMEs

Sixteen files, 506 lines. Five are genuine and useful: the root one, `crates/README.md`, `crates/gpurec-preprocess/src/README.md`, `gpurec/core/README.md`, and `gpurec/core/kernels/README.md` — the best of them. Four are nine-line stubs that restate the directory name and end with the same filler sentence about `--pycache--` being an interpreter artefact.

Table 16: README coverage against size. Line counts are Python only.

Directory	Files	Lines	READMEs
<code>gpurec/core/</code>	21	9,554	6
<code>gpurec/solver/</code>	13	3,572	none
<code>gpurec/fit/</code>	9	1,942	none
<code>gpurec/api/</code>	6	1,942	1
<code>gpurec/config/</code>	6	470	none
<code>gpurec/cli/</code>	5	291	none

The root README’s repository map at `README.md:121-134` never mentions `fit/`, `solver/`, `cli/` or `config/`.

Stale claims in the root README. The headline example fails (lines 47–66); a solver setting `e_init` is documented (line 111) that does not exist; a `notebooks/` directory is advertised (line 134) that is not here; the dependency list (line 138) omits `scipy`, which `pyproject.toml:20` requires; external reference projects are described (lines 158–160) that do not exist; and the `pytest` advice (lines 162–164) concerns tests and vendored directories that are both absent. Separately `gpurec/README.md:9-20` lists six public exports where there are seven.

6.6 The command-line tool is undocumented

`pyproject.toml:29` installs a `gpurec` command with two subcommands, `reconcile` and `fit`. No README mentions it. Its input formats — a directory, a glob, or a family listing file — and its output format are findable only by reading source, as is the fact that it prints in nats while the library works in bits.

6.7 What a new user cannot work out

In rough order of how badly it blocks them: the gene leaf naming rule; that `gene.map` does nothing; the order of the three rates (duplication, loss, transfer — stated exactly twice in the whole repository, at `cli/reconcile.py:31` and `fit/global_fit.py:3`, so anyone reading the parameters directly will guess wrong five times in six); that the parameters are base-2 logarithms; that the loss is in bits; the entire command-line surface; that a `--config` file exists at all; the three undocumented environment variables; and the vocabulary, for which this document’s own glossary is offered as a starting point.

7 Are the naming conventions consistent?

Yes, mechanically — this is not a problem area. A syntax-tree scan over the 63 Python files under `gpurec/` found zero classes not in `CapWords`, zero module-level constants not in upper case, and only 15 of 358 functions not in lower-case-with-underscores — 13 of those being deliberate mathematical shorthand for small local closures. Cross-file naming for the same concept is consistent: the species tree is called `species_tree` in all 18 places and never anything else.

There is one real problem, and it is the same one the review’s own glossary exists to answer.

7.1 Short names nobody has written down

Table 17: Frequently-used short names and where, if anywhere, the repository defines them.

Name	Meaning	Uses	Defined in the repository
S	number of species	114	once, <code>wave_backward.py:706</code>
W	wave width	22	nowhere
G	number of families	17	nowhere
C	number of clades	11	nowhere
sv	forward-solve results	41	nowhere (called saved in one docstring)
Pi, Pibar, E, Ebar	see glossary	700+	informally, in the README's prose
DTS	duplication, transfer, speciation	35	once, <code>core/inference/README.md:7</code>
vjp, jvp	see glossary	many	nowhere
th, al, gp, c1, c2, lam	—	110	nowhere

On top of these are invented step labels with no key: **S3** through **S9** appear in ten comments across `solver/hvp/`, and `solver/curvature/receiver.py:1` opens with “S9: gauge-projected joint...”. None is defined anywhere. Likewise `proposal0_memory_gate`, exported from `core/memory_policy.py` — “proposal0” appears nowhere else.

Three abbreviations are handled correctly and show the authors know how: “second order” is spelled out in line 1 of each `_so` file, “Hessian-vector product” in `solver/hvp/__init__.py:1`, and “Gauss-Newton” in `solver/hvp/gauss_newton.py:1`.

One collision: `so` means “second order” in file names but is a local variable holding a settings object in three files.

7.2 Two names that mislead

`gpurec/optimization.py` versus `gpurec/fit/optimize.py`: the first is rate-bound projection (96 lines), the second is the optimiser loop (561 lines), and nothing in either name distinguishes them.

The `*_fit.py` suffix covers four of the eight fitting modules but not `baselines.py`, `map_cv.py`, `newton_cg.py` or `optimize.py` — and `dtl_fit.py` breaks the pattern's reading, being the dispatcher across modes rather than a mode.

8 Is the directory tree well organised?

The shape is good. Two settings classes are in the wrong package, and that alone explains every layering problem below.

8.1 The intended layering, and where it is violated

Reading the package descriptions, the intended stack is:

`config → core → solver → api → fit → cli`

reading left to right as “may be imported by”. Settings sit at the bottom and know about nothing; the command-line tool sits at the top and may use anything. An arrow pointing right-to-left is a violation.

Table 18: Imports between packages. Numbers are counts of import statements; imports within a package are excluded. A dagger marks a direction that violates the layering above.

Package	Imports from
api	core 15, config 4
cli	config 4, fit 2, api 1
config	solver 4 [†] , api 2 [†]
core	config 2, api 2 [†]
fit	solver 17, config 7, api 5, core 1
solver	core 26, config 8, api 5 [†]

Every violating edge is one of two imports:

- `config` → `api` (2 statements) and `core` → `api` (2 statements) are *all four* the same import: `SolverOptions` from `gpurec/api/solver_options.py`. That file is 58 lines — one dataclass and a validation method — with **zero** imports from `gpurec`. It is pure settings sitting in the model package.
- `config` → `solver` (4 statements, two of them comments) is entirely `PenaltyOptions` from `gpurec/solver/penalties.py` — again a settings dataclass, this time in the solver package.

Moving both into `gpurec/config/` removes every violating edge on the branch. That is item 5.

8.2 The workaround, and a trap in removing it

`PenaltyOptions` causes a genuine circular import. The settings module cannot import it at the top of the file, because doing so pulls in the solver package, which pulls in the model package, which imports the settings module again while it is still half-built. The workaround is to defer the import inside a function, and it carries a nineteen-line comment at `gpurec/config/gpurec_config.py: 21-39` explaining exactly this.

An earlier draft of this review attributed that comment to `SolverOptions`. That was wrong, and the comment itself says so, ending: “`SolverOptions` has no such cycle (`gpurec.api` is a plain namespace package with no `__init__.py`, and `solver_options.py` has no `gpurec` imports of its own), so it stays a normal top-level import.”

That sentence is worth reading twice, because it sets a trap. Section 8.3 notes that six package directories have no `__init__.py` and calls it an inconsistency. But the missing `__init__.py` in `api/` is precisely what stops the `SolverOptions` edge from becoming a real cycle. **Adding an `__init__.py` to `api/` — exactly the tidying that Section 8.3 invites — would create a second circular import**, unless `SolverOptions` is moved out first. Hence the ordering note on item 5.

8.3 Six package directories have no `__init__.py`

They are `gpurec/api/`, `gpurec/core/`, and the four `core/` subpackages.

This is **not** a packaging break. A wheel was built to check: all six subpackages ship (68 entries) and both `import gpurec.api.model` and `import gpurec.core.kernels.pi_forward` succeed, because the packaging tool discovers namespace packages by default.

The cost is intent. `solver/` and `config/` each declare an export list; `api/`, which the root README calls “the stable interface”, declares nothing, so there is no single place saying what it offers. Given Section 8.2, the right response is not to add the files but to move the settings classes first and then decide.

8.4 The private-module convention is not enforced

`gpurec/api/` marks four modules private with a leading underscore. Six files outside `api/` import from them:

```
gpurec/solver/value_and_grad.py:24    from gpurec.api._batch_state    import
    _BatchStatic
gpurec/solver/value_and_grad.py:25    from gpurec.api._execution    import
    stream_batches
gpurec/solver/curvature/genewise.py:6 from gpurec.api._execution    import
    evaluate_static_loss_grad, stream_batches
gpurec/solver/hvp/exact.py:18          from gpurec.api._implicit_grad import
    _neumann_e_adjoint, _safe_exp2_ratio
gpurec/solver/hvp/gauss_newton.py:22   from gpurec.api._implicit_grad import
    _safe_exp2_ratio, implicit_grad_loglik_vjp_wave
gpurec/fit/map_cv.py:25                from gpurec.api._execution    import
    stream_batches
```

Two of these import a private function inside a private module, from another package. `stream_batches` is used by three packages, so it is shared infrastructure living in a file marked private. Either the underscore comes off and the file moves somewhere shared, or the convention is dropped; as it stands it tells the reader something untrue.

8.5 fit/ and solver/ overlap

The stated split — `solver/` is reusable mathematics, `fit/` is per-mode recipes — reads well but is not held:

- A mode-specific recipe lives in `solver/`: `solver/curvature/genewise.py`, 456 lines named after a fitting mode, alongside `fit/genewise_fit.py`.
- Newton drivers live in both, and `fit/newton_cg.py`:82 imports one from `solver/curvature/receiver.py`, so a fitting driver calls a solver driver.
- Generic machinery lives in `fit/`: `fit/optimize.py` describes itself as “dataset-agnostic”.

`fit/dtl_fit.py` is the exception and the model to follow — its docstring claims to be “the ONLY place that choice is made”, which is the right idea.

`api/` versus `core/` is defensible: 6 files of model wiring against 21 files of kernels and scheduling. The only blur was the two imports resolved in Section 8.

8.6 The two loose files at the package root

`gpurec/optimization.py` (96 lines) is load-bearing but homeless: it is imported by `gpurec/__init__.py`:4-6 and two fitting modules, and it pairs with `gpurec/config/rates.py`, whose own docstring names it. It belongs beside that file.

`gpurec/distributed.py` (112 lines) has zero importers anywhere and is already counted in Section 5.3.

8.7 The public surface, and one packaging gap

`gpurec/__init__.py` is 7 lines exporting 7 names. One of them, `sample_reconciliations`, is pulled straight from a `core` internal with no intermediate export point.

`gpurec/config/defaults.toml` is **not shipped in the wheel** — there is no packaging entry for it, confirmed by building one. The practical impact is small, since that file is never read at runtime anyway, but it means the drift check described at `gpurec/config/gpurec_config.py`:200-202 cannot be run against an installed copy.